Debate2Rescue

Final Report for the Intelligent Systems Engineering Course 2025/2026

Nicolas Amadori
nicolas.amadori@studio.unibo.it

Riccardo Mazzi
riccardo.mazzi@studio.unibo.it

Denise Nanni
denise.nanni@studio.unibo.it

October 30, 2026

When several emergencies need the same scarce rescue resources, a priority score picks one quickly but hides why it was preferred, and gives no way to argue for an exception. Debate2Rescue is a multi-agent system that allocates these resources through argumentation-based negotiation. Rescuer agents reach the emergencies that appear on a simulated grid and request the specialised units they need, which pilot agents then bring to the scene, while an orchestrator dispatches the rescuers and runs the debates. The rescuers take turns revealing facts about their emergency, such as its severity or how often it has been denied, and withdraw when nothing would improve their position. After every move, a shared rule base written in Arg2P is evaluated under grounded semantics to find the request that currently wins. Repeated denials weigh more than any single attribute, so waiting requests gain ground, but dangerous situations count as reasons of their own, and a critical patient outweighs even the denials. The agents are written in Jason, and every debate is logged with the arguments that decided it.

AI Disclaimer

During the preparation of this work, the authors used LLMs to assist during development, fix grammatical errors in the report, and improve clarity.

After using these tools/services, the authors reviewed and edited the content as needed and take full responsibility for the content of the final report and artifact.

1 Introduction

When several emergencies break out in the same area, the rescuers handling them sometimes need the same scarce resources, such as an ambulance or a helicopter, and someone has to decide who is served first and who waits.

A simple way to make this decision is to reduce every request to a number, like a fixed priority score based on the emergency data of each rescuer, and to serve the highest one. It is fast, but the number alone does not show which aspect of the emergency produced it, and the rescuers who handle an exception cannot argue for it. Consider two medical emergencies that both want the only free ambulance: the first has already been turned down more times, while the second involves a critical patient and the first does not. A score that weighs waiting time heavily, as one would to keep requests from being starved, sends the ambulance to the first one, meanwhile the person in critical condition keeps waiting and the second team cannot argue that being critical matters more than having waited longer, so the system has no way to weigh such an argument against the others.

Debate2Rescue explores *argumentation-based negotiation* [Rahwan et al., 2003] as a way to make those decisions, asking agents to back their requests with reasons drawn from their emergency, such as its severity or how many times the request has already been denied. When several agents want the same resource, they debate over multiple rounds, challenging each other on these reasons and withdrawing when nothing they can add would help. A shared set of domain rules, evaluated after every move, decides which request currently prevails. In the example above, the rules give the ambulance to the second emergency even though it has waited less, and the arguments that won the debate explain the choice.

The project is a multi-agent system that simulates an area where natural and human-caused emergencies arise: a rescuer agent is sent to each emergency and, once on site, requests the resource it needs, which a pilot agent then brings to the scene. A central orchestrator dispatches the rescuers, mediates resource conflicts, and it starts a debate only when there are not enough units to satisfy every request. All agents are BDI agents written in Jason, and the debates are evaluated with the Arg2P argumentation framework.

Section 2 gives an overview of the literature the project builds on, while section 3, section 4, and section 5 describe the system and how it was built. Section 6 explains how to run the project, and section 7 explores possible future work.

2 Literature Review

2.1 Abstract and Structured Argumentation

Argumentation draws conclusions from incomplete or conflicting information by weighing arguments for and against a claim. Most formal work builds on the abstract frameworks of Dung [1995], where arguments have no internal structure and a framework is a set of arguments with a binary attack relation. A semantics then selects the sets of arguments, called extensions, that can be accepted together. A set defends an argument when it

attacks all of its attackers. We use the grounded semantics, a skeptical one that starts from the unattacked arguments and repeatedly adds those defended by the arguments accepted so far. As a result, two arguments that attack each other are both left out unless another accepted argument defends one of them. Unlike semantics that admit several alternative extensions, the grounded one always yields exactly one, so each evaluation of a debate has a single result.

Structured argumentation looks inside the arguments, building them from strict and defeasible rules, so that attacks follow from their structure. An argument can contradict the conclusion of another one (rebutting), one of its premises (undermining), or deny that one of its defeasible rules applies (undercutting) [Modgil and Prakken, 2014]. Our debates rely on rebuttals and undercutters, while undermining plays no role.

Arg2P [Calegari et al., 2022] is an ASPIC+-like system built on top of 2P-Kt. Rules are written in an enriched Prolog syntax and preferences between them are stated through a superiority relation. The resulting graph is evaluated with a labelling semantics, grounded in our case, which marks each argument as accepted (`IN`), rejected (`OUT`) or undecided (`UND`). Its authors present it as a lightweight library, meant to be embedded in a larger system and called only where reasoning is needed. Since it is built on logic programming, its conclusions can be traced back to readable rules, increasing explainability. In our project, Arg2P is used by the orchestrator to evaluate the ongoing debate, and by the rescuers to pick their moves.

Pisano et al. [2021] make preferences *defeasible* in Arg2P, so that they are themselves conclusions of arguments and can be attacked, meanwhile in this project preferences are fixed, and context-dependent situations are modelled as extra reasons in the same hierarchy rather than as arguments about preferences.

2.2 Argumentation in Multi-Agent Systems

Calegari et al. [2022] observe that the abstract argumentation perspective tends to see argumentation as a one-sided process in which a single party justifies a claim, while in practice it is usually an exchange between several contributors, as in multi-agent systems. Rahwan et al. [2003] survey *argumentation-based negotiation*, where agents with conflicting interests, typically competing for scarce resources, exchange arguments that justify their stance or try to influence the others. They contrast it with heuristic approaches, in most of which agents can only exchange proposals and accept or reject them. Their conceptual framework fits our design, with custom performatives as the communication language, the orchestrator's turn-taking as the protocol, and the agents' beliefs as information stores. Among the open problems, the survey asks whether a mediator should merely assist the negotiators or be able to enforce its will. Our orchestrator operates as an in-between, as it runs the turns and hands out the resource, but the winner follows from the shared rules and the other rescuers' withdrawals.

2.3 BDI Agents and Jason

The belief-desire-intention (BDI) model describes an agent through what it believes about the world, the goals it wants to achieve, and the plans it commits to. In this project we use Jason Bordini et al. [2007], an interpreter for an extended version of AgentSpeak. Jason agents react to changes in their beliefs and goals by selecting plans from a plan library, and communicate through speech-act messages whose performatives, such as `tell` or `achieve`, define the effect on the receiver. This effect is itself defined by AgentSpeak plans, which can be overridden to handle new performatives. The language can also be extended with internal actions, Java classes that can be called from plans and rules, where they behave like predicates, serving as our bridge to Arg2P and to the environment.

3 Analysis

3.1 Problem Domain and Motivation

Urban disaster response requires managing dynamic emergencies across a 2D grid containing obstacles, base stations, and mobile responders. The primary engineering challenge is *resource contention*: while field rescuers perform initial on-site triage, resolving incidents requires scarce specialized units (such as fire trucks, ambulances, or helicopters) that competing emergencies must share.

Standard allocation strategies like First-Come-First-Served or static scoring either ignore urgency or rely on opaque numerical formulas. Instead, Debate2Rescue models allocation as an argumentation process. Rescuers evaluate local incident conditions (such as severity, victim count, or trapped individuals) and deliberate using Arg2P. Formal domain rules and situations resolve conflicts under grounded semantics, assigning the contested resource to the most justified claim.

3.2 System Roles and Responsibilities

The system identifies four primary roles, each associated with specific duties:

1. **Orchestrator Agent:** Coordinates the overall response, dispatching available rescuers to newly detected incidents, collecting incoming resource requests, moderating argumentative debates when resources are contested, and granting units to the winning emergency.

2. **Rescuer Agent:** Acts as an on-site first responder, traveling to the incident scene, assessing local conditions, requesting the required specialized equipment, and defending the emergency's claim during resource debates.

3. **Pilot Agent:** Operates specialized vehicles and equipment, deploying to an incident when assigned, executing the physical rescue operation until the emergency is resolved, and returning to base.

4. **Simulation Operator (User):** The human supervisor who oversees the simulation, can configure the initial environment, adjust simulation parameters at runtime, and monitor agent behavior and debate outcomes.

3.3 Functional Requirements

The functional requirements describe the core features and operations that the system must support:

- **Environment Simulation:** The system simulates a 2D grid world containing obstacles, stations, and emergencies with different categories and different attributes. The simulation spawns new emergencies based on user-defined parameters.
- **Emergency Dispatch:** The orchestrator automatically assigns the nearest available rescuer to new emergencies. The rescuer navigates to the incident cell, inspects local conditions to discover attributes, and requests the appropriate specialized resource.
- **Request Window:** When resource requests arrive, the orchestrator collects them within a short time window. If the number of competing requests exceeds available units, a debate is opened; if units are sufficient, they are granted directly.
- **Argumentation-Based Deliberation:** Contending rescuers participate in a turn-based dialogue using custom performatives. Moves are evaluated via Arg2P using grounded semantics to determine the winning emergency based on explicit domain rules and situations.
- **Rescue Execution and Agent Lifecycle:** The assigned pilot travels to the incident and executes the rescue. Once resolved, the emergency is removed, and both responders return to base to become available again. Denied contenders retain their requests for subsequent allocation rounds.
- **GUI and Runtime Control:** The interface provides a live visual grid with tooltips and dynamic sliders that allow the operator to adjust simulation speed, spawn rate, and emergency limits during execution.
- **Audit Logging:** The system logs all simulation events in real time and automatically exports complete debate transcripts to disk, recording every move, intermediate status, and the final justification.

3.4 Non-Functional Requirements

The non-functional requirements define the quality attributes and operational constraints of the application:

- **Explainability:** Every allocation decision must provide an explicit, rule-based chain of arguments, clearly showing which criteria or situations decided the outcome.

- **Fairness:** Lower-priority emergencies must not be starved indefinitely. Each denial increases an emergency's accumulated waiting weight, allowing older requests to eventually take precedence over higher-severity but newer non-critical incidents.
- **Performance:** Deliberation through Arg2P, pathfinding, and visual updates must execute with minimal latency, ensuring a smooth simulation loop and a responsive user interface.
- **Concurrency:** State changes, such as rescuer assignments and resource grants, must be thread-safe and atomic to prevent race conditions across parallel agent threads.
- **Observability:** The interface must make system behavior easy to understand and follow.
- **Portability:** The project must run reliably across major desktop operating systems, without requiring manual external dependencies.

4 Design

4.1 Architecture

The system consists of a multi-agent architecture designed to separate the physical simulation environment, the BDI reasoning loop, and the argumentation engine.

Agent-environment interaction follows the standard perception-action loop: agents receive environmental percepts and perform physical actions. Agent coordination is realized through asynchronous messages using both standard and custom performatives designed for argumentation dialogues.

The architecture consists of five main layers:

1. **Environment and Simulation Layer (Java):** Composed of the simulation environment (`EmergencyEnvironment`) and the domain model (`SimModel`), this layer manages the 2D grid, obstacles, stations, and emergencies. A dedicated thread generates emergencies according to the selected spawn rates, while the environment executes agent actions and updates agent percepts.
2. **MAS Infrastructure Layer (Jason):** This layer is based on the centralized Jason infrastructure declared in `debate2rescue.mas2j` and manages the execution cycle of BDI agents, scheduling plan intentions and delivering messages through the agent communication channel.
3. **Agent Layer (AgentSpeak):** Defines the autonomous entities that operate in the system: the Orchestrator, Rescuers, and Pilots. Agents maintain local beliefs, react to environment percepts, and coordinate through communication protocols.
4. **Argumentation Layer (Arg2P):** Implements argumentation-based debates using Arg2P. It evaluates argumentation theories defined in Prolog and, through the use of custom Jason internal actions, integrates the results with the BDI agents.

5. **Observability and User Interface Layer (Swing):** It provides a real-time visual representation of the grid, dynamic controls for simulation parameters, an interactive debate arena, and an audit logger that persists debate transcripts to disk.

Figure 1 illustrates the overall system architecture and the relationships among the five layers, highlighting the communication channels and data flow between the Java simulation environment, the Jason BDI runtime and agents, the Arg2P argumentation engine, and the Swing graphical user interface.

4.2 Detailed Design

4.2.1 Operational Workflows

The overall response cycle proceeds through four sequential phases:

1. **Emergency Detection and Dispatch:** When an emergency appears on the grid, the orchestrator receives the corresponding percept. It identifies all idle rescuers, calculates their grid distance to the incident, and assigns the closest one. The assigned rescuer navigates step by step toward the incident location.

2. **On-Site Triage:** Upon reaching the emergency cell, the rescuer invokes `start_rescue` to uncover the specific attributes of the incident: severity level, victim count, trapped individuals, escalating danger, physical size, and collateral risk. Using a local belief table, the rescuer identifies the required resource and sends a request to the orchestrator.

3. **Request Window and Contention Detection:** To avoid greedy, first-come-first-served allocations, the orchestrator groups incoming requests within a short time window. Once the window closes:
   - If the number of competing requests for a given resource type is less than or equal to the number of free units, the orchestrator grants the resources directly.
   - If requests exceed the available units, resource contention occurs and the orchestrator starts a new debate.

4. **Rescue Execution:** The pilot assigned to the winning emergency travels to the site. The time required to resolve the emergency is proportional to the incident difficulty. Once resolved, the emergency is cleared from the environment, and both the rescuer and the pilot return to their base station, informing the orchestrator that they are free. Rescuers whose claims were denied remain on-site and increment their denial counter, gaining priority in subsequent allocation rounds.

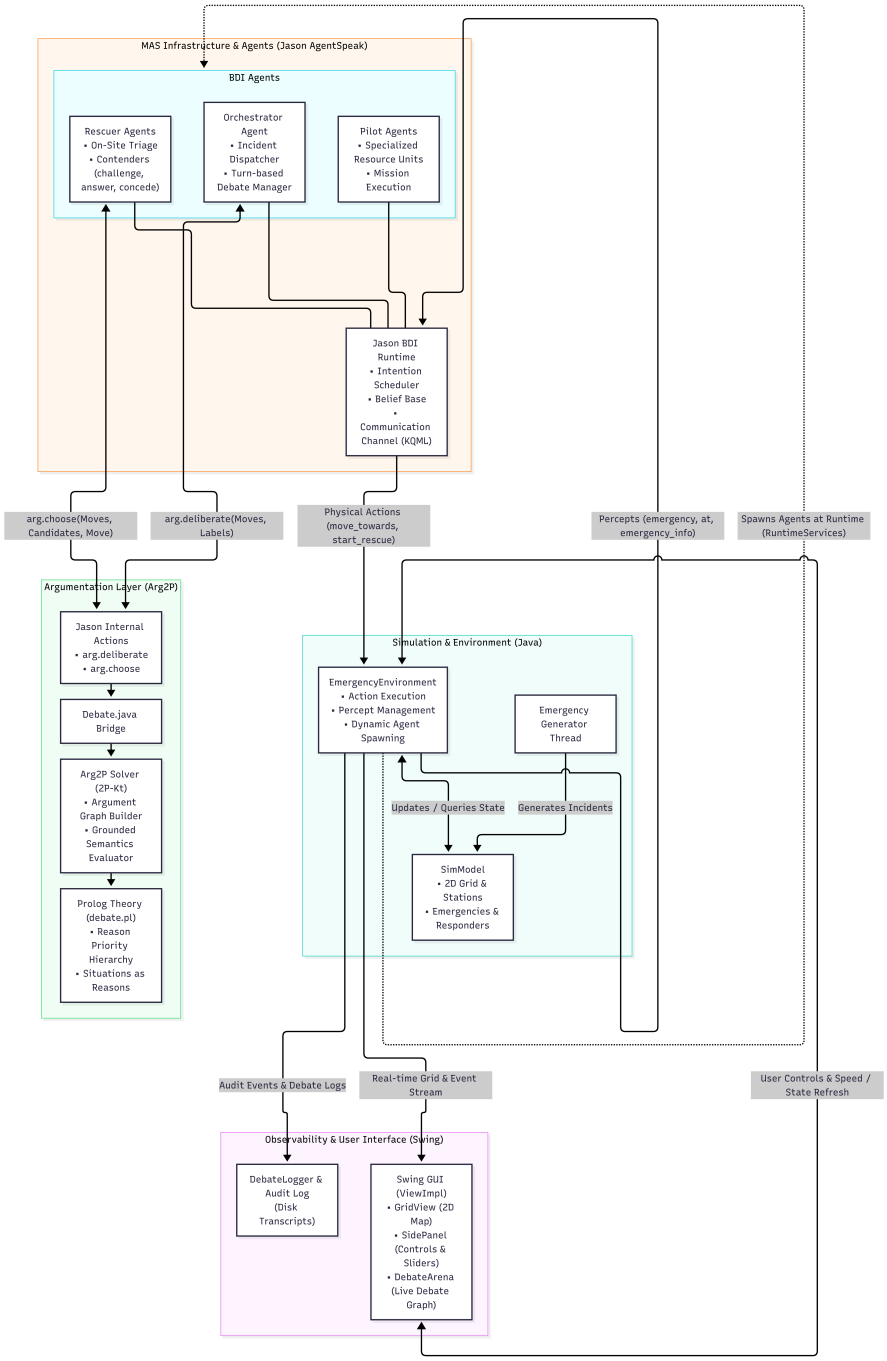


Figure 1: System architecture diagram depicting the interaction between the Java simulation environment, the Jason BDI runtime and agents, the Arg2P argumentation engine, and the Swing GUI.

4.2.2 Argumentation Model

Resource allocation disputes are solved through structured argumentation rather than numerical scoring functions.

The argumentation theory models claims, evidence, principles, and preference rules:

- **Claims:** Contenders assert a claim for the resource (`claim(E)`). Initial claims temporarily grant the resource based merely on the emergency id E, oldest first.
- **Challenge:** Revealing a value for a reason R puts an emergency ahead of every contender that has not revealed its own value for R. The worst values, such as ESI 5, cannot be used this way.
- **Compare:** When two contenders have presented their values for the same reason R, they are automatically compared and the better value puts its emergency ahead. Being ahead ranks an emergency over the other, and an emergency ranked below someone loses its claim. Opposite rankings are settled by the strength of their reason R:

  $$\texttt{critical} > \texttt{waiting} > \texttt{catastrophic} > \texttt{major_disaster} > \texttt{spreading} > \texttt{severity} > \texttt{collateral_risk} > \texttt{trapped} > \texttt{escalating} > \texttt{size} > \texttt{victims}$$

  Accumulated waiting time (the number of prior denials) outranks everything except a critical situation, to prevent starvation.
- **Situations:** Some values, alone or combined, define a dangerous situation, which is a reason placed in the same hierarchy (formalized in table 1). A single generic rule puts an emergency that is in a situation ahead of a contender that is not:
  - `critical`: The incident has severity ESI 1 or 2.
  - `catastrophic`: The incident presents collateral risk at medium or large scale.
  - `major_disaster`: The incident is large and has many victims.
  - `spreading`: People are trapped and the incident is escalating.
- **Tie-Breaking:** If no reason sets two emergencies apart, the older emergency ID wins.

4.2.3 Custom Performatives

Standard agent communication languages provide general-purpose performatives, but lack speech acts dedicated to argumentation. We extend agent communication in Jason by introducing four custom performatives handled via the `+!kqml_received` pattern:

- `argue`: Sent by the orchestrator to all contending rescuers to formally initiate a debate and broadcast the contenders' initial claims (`move(RT, M, E, claim)`) for the contested resource type.

- `challenge`: Sent by a rescuer on its turn to the orchestrator (and relayed to all contenders) to attack a target competitor's claim by revealing an emergency attribute and its value (`challenge(RT, Target, Reason, Value)`) where the attacker holds an advantage.

- `answer`: Sent by a challenged rescuer to the orchestrator (and relayed to all contenders) to defend its claim against a specific challenge move ID (`answer(RT, M, Reason, Value)`), either by revealing a superior value on the disputed reason or by disclosing attributes that complete a situation.

- `concede`: Sent by a rescuer to the orchestrator (`concede(RT)`) when Arg2P finds no admissible move that improves its position. The orchestrator removes the rescuer from the active turn rotation, notifies remaining contenders to retract the conceded claim, and marks the request as denied for the current allocation round.

The end-to-end turn-based debate protocol between the Orchestrator and contending Rescuers is illustrated in Figure 2. When resource contention occurs at the closure of a request window, the Orchestrator opens a debate and broadcasts the initial claims via `argue`. In subsequent turns, agents evaluate their local knowledge through Arg2P, issuing attacks (`challenge`) or defenses (`answer`), which the Orchestrator records and relays to maintain synchronized debate models across all contenders. Once a contender cannot construct an improving move, it emits `concede`, allowing the Orchestrator to conclude deliberation and dispatch the disputed resource to the winner.

4.2.4 Internal Actions

Two custom Jason internal actions integrate Arg2P into the agents' reasoning cycles:

- `arg.deliberate(Moves, Labels)`: Executed by the orchestrator. It builds the argumentation theory from the move history, evaluates the graph using grounded semantics, and returns the status of each claim (`in`, `out`, `und`) along with the active winning justification (`winning_reason`).

- `arg.choose(Moves, Me, Other, Candidates, Chosen)`: Executed by rescuers on their turn. It selects the first candidate that strictly improves its position against the target competitor, searching first one move ahead and then the combinations of two moves. If no candidate improves its position, the internal action fails, prompting the rescuer to send a `concede` move.

4.3 Agent Modeling

4.3.1 Orchestrator Agent

- **Beliefs:** Tracks detected emergencies (`emergency`), idle rescuers (`availableRescuer`), available resource units (`availableResource`), active collection windows (`window`), and debate states (`debate`, `debate_id`, `move`, `turn`).

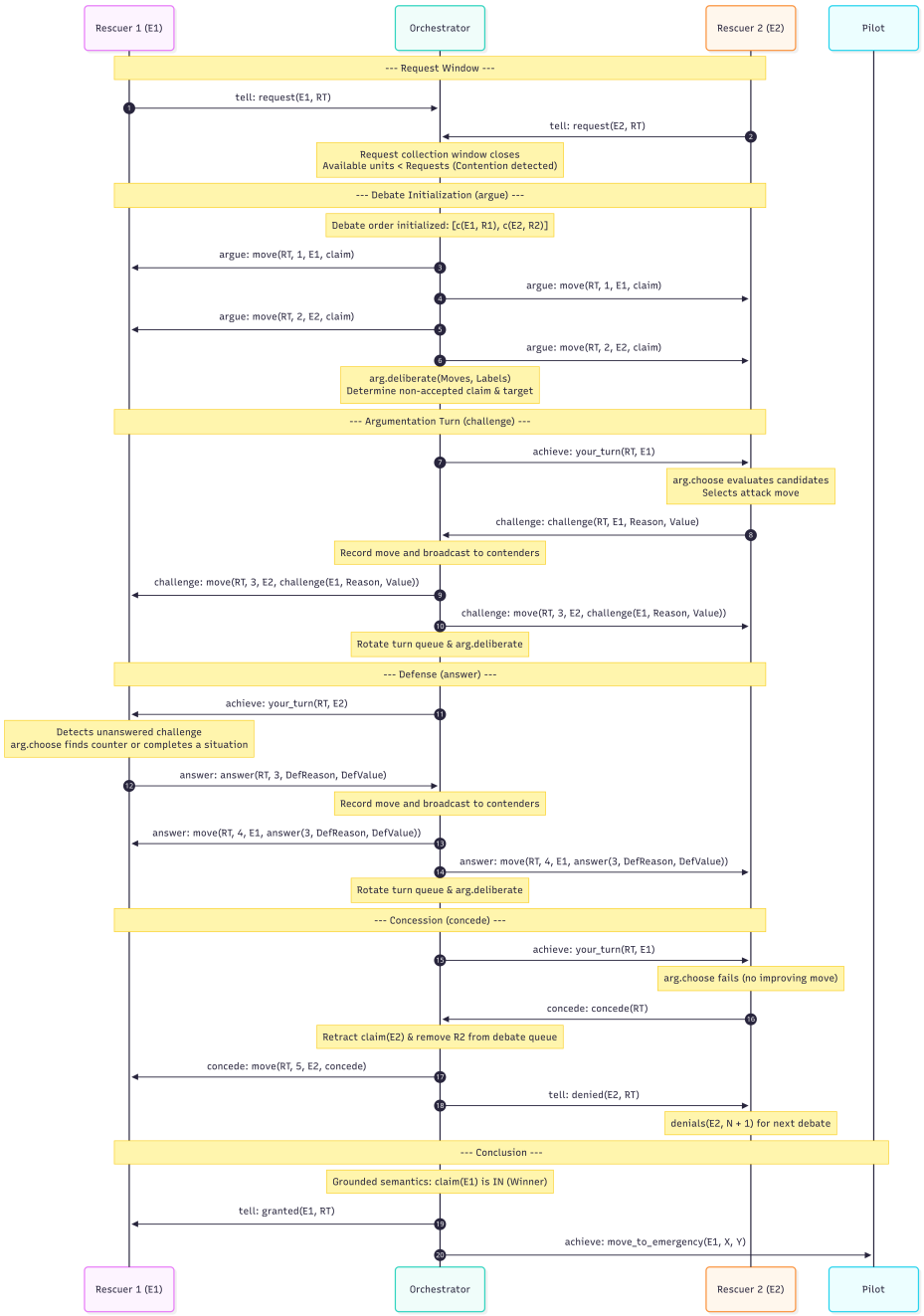


Figure 2: Sequence diagram illustrating the turn-based debate dialogue between the Orchestrator and competing Rescuers, covering request contention, claim initialization (`argue`), targeted attack (`challenge`), defense (`answer`), concession (`concede`), and resource dispatch.

- **Plans:**
  - `!assign`: Assigns available rescuers to new incidents, prioritizing the closest rescuer.
  - `!close_window`: Closes the request collection window and initiates resource allocation or debate.
  - `!deliberate`: Evaluates whether a debate is needed or if resources can be granted directly.
  - `!next_turn` and `!give_turn`: Coordinate turn rotation among contenders whose claims are not accepted under grounded semantics.
  - `!grant`: Dispatches the nearest available pilot to the winning incident.
- **Concurrency:** Critical plans handling resource state and debate updates are declared as atomic to ensure thread safety when concurrent events occur simultaneously.

4.3.2 Rescuer Agent

- **Beliefs:** Stores the assigned emergency ID and location, perceived emergency facts, the required resource type (`requesting`), the accumulated denial count (`denials`), and the debate moves history.
- **Plans:**
  - `!handle`: Receives an incident assignment, navigates to the target cell, and performs triage.
  - `!evaluate` and `!make_request`: Identifies the necessary specialized resource and submits the request to the orchestrator.
  - `!your_turn`: Activated when the orchestrator prompts the rescuer to speak. It evaluates candidate defenses or attacks via `arg.choose` and emits an `answer`, `challenge`, or `concede` message.
  - `!go_home`: Returns to the base station after emergency resolution and announces availability.
- **Strategy:** Rescuers disclose emergency attributes incrementally only when required to counter an opponent's argument, minimizing communication overhead.

4.3.3 Pilot Agent

- **Beliefs:** Maintains the assigned base station coordinates, its specialized resource type, and its availability status.
- **Plans:**
  - `!start`: Initializes the agent in the environment and informs the orchestrator of its readiness.

– `!move_to_emergency`: Dispatched by the orchestrator upon winning a debate. The pilot navigates to the incident, simulates the rescue operation duration ($2 \times$ severity + victims), resolves the emergency, signals the waiting rescuer, and returns to base.

5 Implementation

Debate2Rescue is a Gradle project in Java 21 built on the Jason interpreter (3.2.1) and the JVM distribution of Arg2P.

5.1 Environment and Model

`EmergencyEnvironment` creates the rescuers and pilots at runtime, in the number chosen in the configuration dialog (section 6.2), and runs the simulation loop that spawns emergencies on the grid, an extension of Jason's `GridWorldModel`. Emergency IDs come from an incremental counter, so a lower ID always means an older emergency, and both the orchestrator and the theory use this to break ties.

In the simulation model, an incident is represented by an immutable `Emergency` record containing the full set of emergency information:

- **Identity and location:** A unique integer `id` and its grid coordinates `position` (x, y).
- **Type:** An `EmergencyType` enum describing the nature of the incident (e.g., `FIRE`, `MEDICAL`, `HAZMAT`, `BOMB`, `AGGRESSION`, `EVACUATION`, `ARMED_PERSON`, `DISASTER`, or `MISSING`), each uniquely mapped to a specialized vehicle or unit type (such as fire trucks, ambulances, or helicopters).
- **Severity:** An integer on the Emergency Severity Index (`Severity`, ESI 1 to 5), ranging from 1 for immediately life-threatening incidents to 5 for non-urgent cases.
- **Victim count:** The number of affected persons requiring assistance (`victims`, from 0 to 15).
- **Hazard conditions:** Boolean flags denoting whether individuals are trapped (`trapped`), whether the incident is worsening over time (`escalating`), and whether adjacent critical infrastructure is threatened (`collateralRisk`).
- **Scale:** A physical magnitude (`size`) rated as small (1), medium (2), or large (3).
- **Status:** An operational state (`EmergencyStatus`), which transitions from `WAITING` to `IN_PROGRESS` when the assigned pilot intervenes, and finally to `RESOLVED` after completion.

Jason runs environment actions on several threads, so the model's methods on emergencies and responders are `synchronized`.

5.2 The Debate Theory

The domain rules shared by all agents are in `arg/debate.pl`, they never change during a run and the Java side only appends facts to them: `contender(E)` for each emergency still claiming the resource and `val(E, Reason, Value)` for each revealed value. Figure 3 shows the core rules, without the helper predicates and the situations, which are summarised in table 1.

```
claim(E) : prolog(contender(E)) => grant(E).
exclude(E1, E2) : grant(E1), prolog(contender(E2)), prolog(E1 < E2) -> -grant(E2).

compare(R, E1, E2) : val(E1, R, V1), val(E2, R, V2), prolog(beats(R, V1, V2))
    => ahead(R, E1, E2).
challenge(R, E1, E2) : val(E1, R, V1), prolog(worth(R, V1)), prolog(contender(E2)),
    prolog(E1 \== E2) => ahead(R, E1, E2).
answer(R, E2, E1) : val(E2, R, V2), prolog(contender(E1)), prolog(E1 \== E2)
    => undercut(challenge(R, E1, E2)).

rank(R, E1, E2) : ahead(R, E1, E2) => over(E1, E2).
asym(E1, E2) : over(E1, E2) -> -over(E2, E1).
beaten(E1, E2) : over(E1, E2) => -grant(E2).

sup(beaten(_, _), claim(_)).
sup(rank(R1, _, _), rank(R2, _, _)) :- stronger(R1, R2).
sup(claim(E1), claim(E2)) :- contender(E1), contender(E2), E1 < E2.
```

Figure 3: Core rules of `debate.pl`. The situations and the helper predicates are omitted.

Each contender has an initial claim to the resource, and a granted emergency excludes the younger ones, so with claims alone the oldest one wins. An emergency gets ahead of another on a reason through `challenge`, `compare`, or `in_situation`. A revealed value that counts as a reason, according to the helper `worth`, challenges every other contender at once. The worst values, such as severity 5 or size 1, do not count. The challenged one undercuts it by revealing its own value for the same reason, and from then on `compare` decides between the two values. Being ahead turns into a ranking, and opposite rankings rebut each other, with the stronger reason prevailing. As defined in section 4.2, a critical situation is the strongest reason, followed by waiting, the other three situations, severity, collateral risk, trapped victims, escalation, size, and victims; waiting beats every other attribute and every situation except a critical one, which satisfies the fairness requirement (section 3.4). An emergency with someone ranked over it loses its claim, and the arguments that used that claim to exclude younger contenders fall with it, which lets a younger emergency win.

The situations in table 1 are concluded by strict rules from the revealed values, and `in_situation` puts an emergency that is in a situation ahead of a contender not known to be in it. This argument falls once the contender reveals values that complete the same situation.

In the debate logged in fig. 10, emergency 10 takes the lead from the older claim of 7 with a challenge on severity, and its ESI 2 also makes it critical. Emergency 7 is

Situation	Condition	Strength
`critical`	ESI ≤ 2	11
`catastrophic`	collateral risk, size ≥ 2	9
`major_disaster`	size ≥ 3, victims ≥ 8	8
`spreading`	trapped and escalating	7

Table 1: Situations of the theory, with the values that complete them and their strength as a reason.

large with 12 victims, so revealing them would make it a major disaster, which outranks severity but not a critical situation, and 7 concedes.

5.3 Internal Actions

5.3.1 Argumentation Actions

The argumentation reasoning is integrated into Jason via two custom internal actions extending `DefaultInternalAction`, which interface agents with Arg2P through the helper class `Debate`:

- `arg.deliberate(Moves, Labels)`: Executed by the orchestrator. It maps the dialogue move history into Prolog facts, evaluates the theory with Arg2P under grounded semantics, and returns the resulting argument labels to identify contenders and manage turn rotation.
- `arg.choose(Moves, Me, Other, Candidates, Chosen)`: Executed by competing rescuers on their turn. It simulates candidate dialogue moves against an opponent in a pairwise sub-debate and unifies `Chosen` with the first move that strictly improves the agent's standing. It evaluates single moves first, then candidate pairs for situations that need two values, failing if no candidate improves its position.

5.3.2 Environment Actions

Custom environment actions declared in `EmergencyEnvironment` are dispatched by `executeAction` to manipulate `SimModel`, coordinate agent lifecycles, and synchronize telemetry with the GUI:

- `join_as_rescuer`: Registers a newly spawned rescuer agent at an available base station.
- `join_as_pilot`: Registers a newly spawned pilot agent with its specialized resource type at a base station.
- `move_towards(X, Y)`: Advances the agent one step towards grid coordinates (X, Y), respecting simulation speed and obstacles.
- `wait_steps(N)`: Pauses agent execution for N simulation steps.

- `assign(Rescuer, EmergencyId)`: Sets the emergency status to `ASSIGNED` in the model upon rescuer allocation.
- `start_rescue(EmergencyId)`: Transitions the emergency status to `IN_PROGRESS` when the rescuer reaches the incident.
- `resolve_emergency(EmergencyId)`: Marks the incident as resolved, removes it from `SimModel`, and notifies agents of the environment update.
- `log(Message)`: Emits agent log messages to both the console and the GUI log panel.
- `log_debate_start`, `log_debate_move`, `log_debate_status`, `log_debate_end`: Record debate lifecycle events, dialogue moves, provisional rankings, and final resolutions to persist logs via `DebateLogger` and update the GUI debate arena.

5.4 Agents

Requests for the same resource type are buffered by the orchestrator in a window of five steps, run as a separate intention with `!!` so that it keeps reacting while the window is open. When it closes, the orchestrator grants the requests directly if enough pilots of the type are free, and opens a debate if only some are. A rescuer maps its emergency onto the seven values it can reveal, using the number of debates it has lost as its value for waiting. On its turn it runs the plan in Figure 4, where `requesting` holds its request and `moves` its copy of the debate.

Before each turn it evaluates the debate with `arg.deliberate`, and the turn rotates among the contenders whose claim is not IN, each receiving the goal `your_turn(RT, Target)` with an opponent to target. Rescuers reply with the custom performatives `challenge`, `answer`, and `concede`, handled through plans for Jason's `kqml_received` goal that accept a move only from the rescuer holding the turn. Every move is relayed to all contenders, so each rescuer keeps the same copy of the debate as the orchestrator. When nobody is left to give the turn to, the winner gets the closest free pilot, which travels to the emergency and resolves it, while the losers are told `denied` so that they can request again the resource to the orchestrator.

```
+!your_turn(RT, T)[source(orchestrator)]
    : requesting(Id, RT, _) & moves(RT, Moves) <-
    if (unanswered(RT, Id, M, A, R) & answers(RT, Id, M, R, Answers) &
        arg.choose(Moves, Id, A, Answers, answer(M, R2, V))) {
        .send(orchestrator, answer, answer(RT, M, R2, V));
    } elif (challenges(RT, Id, T, Challenges) &
            arg.choose(Moves, Id, T, Challenges, challenge(T, AR, AV))) {
        .send(orchestrator, challenge, challenge(RT, T, AR, AV));
    } else {
        .send(orchestrator, concede, concede(RT));
    }.
```

Figure 4: The plan of `rescuer.asl` that picks the rescuer's move on its turn.

The rescuer first tries to answer a challenge it has not answered yet, offering its value for that reason and then its other unrevealed values, since a different value can complete a situation. Otherwise it challenges its target with its unrevealed values from the strongest reason down, and if nothing helps it concedes.

5.5 Dynamic Number of Agents

To support arbitrary configurations without manual configuration editing, the project separates agent instantiation from the static project file: only the orchestrator agent is statically declared, while rescuer and pilot agents are spawned dynamically at runtime during environment startup based on the configuration chosen by the user.

To ensure scenario validity and operational balance, spatial entities scale with grid dimension M (as detailed in section 6.2), while agent populations are bounded relative to station count S:

- **Rescuers:** Bounded such that $S \leq \text{rescuers} \leq 4 \times S$. This guarantees that every base station has at least one initial rescuer, with a maximum of four rescuers per station.
- **Pilots:** Bounded such that $\max(S, 9) \leq \text{pilots} \leq 4 \times S$. The lower bound of 9 guarantees that each of the nine specialized resource types (fire truck, ambulance, police car, hazmat team, bomb squad, army, excavator, police helicopter, helicopter) has at least one pilot available in the simulation.

Because dynamic agents are created after the orchestrator has started, they register through a standardized startup handshake protocol:

1. Upon instantiation, the agent executes its initial goal `!start`.
2. The agent calls an environment action (`join_as_rescuer` or `join_as_pilot`), which places the agent entity at an available base station in the grid model.
3. The agent sends an informative message to the orchestrator:

   ```
   .send(orchestrator, tell, free(rescuer))
   ```

   for rescuers, or:

   ```
   .send(orchestrator, tell, free(pilot))
   ```

   for pilots.
4. The orchestrator handles these messages dynamically, registering available rescuers (`+availableRescuer(R)`) or pilots (`+availableResource(RT, P)`) into its belief base.

This mechanism allows the system to scale to different population sizes without altering agent source code or restarting the multi-agent infrastructure.

5.6 Logging and Testing

The orchestrator logs every debate through environment actions that forward the events to `DebateLogger`, which writes one file per debate (section 6.6) and updates the side panel. With each status update the orchestrator logs the rank rules that are IN, whose reason can also be a situation, and the logger uses the final list as the justification of the debate or, if it is empty because the others conceded, the last reason seen during the debate.

JUnit tests cover the theory rule by rule, the move choice of `arg.choose`, the environment and the model, and a GitHub Actions workflow runs them, together with a Spotless formatting check, on every pull request toward `main` and `develop`.

6 User Guide

This section describes how to configure, run, and monitor the Debate2Rescue simulation. It details system requirements, startup configuration, the graphical user interface (GUI), runtime controls, and debate log files.

6.1 Getting Started

6.1.1 Prerequisites

The simulation requires:

- **Java Development Kit (JDK):** Version 21 or higher.
- **Gradle:** The repository bundles the Gradle wrapper (`gradlew`), so a separate local Gradle installation is not required.
- **Operating System:** Linux, macOS, or Windows with graphical desktop support.

6.1.2 Running the Simulation

To launch the simulation with default settings, run from the project root:

```
./gradlew run
```

or on Windows:

```
gradlew.bat run
```

On startup, a configuration dialog opens before initializing the agents and grid environment.

6.2 Initial Configuration

The startup dialog (`SimConfigDialog`, Figure 5) allows configuring the grid dimensions and agent population.

The configurable parameters and ranges are summarized in Table 2.

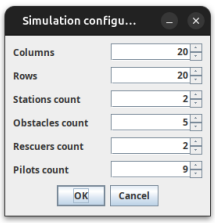


Figure 5: Simulation configuration dialog.

Parameter	Default	Range	Description
Columns	20	10 – 50	Grid width.
Rows	20	10 – 50	Grid height.
Stations	2	1 – MaxStations	Number of base stations.
Obstacles	5	0 – MaxObstacles	Number of obstacle cells.
Rescuers	2	MinRescuers – MaxRescuers	Number of rescuer agents.
Pilots	9	MinPilots – MaxPilots	Number of resource pilots.

Table 2: Initial configuration parameters.

6.2.1 Validation Rules and Default Values

The dialog enforces several constraints based on the average grid dimension $M = (\text{Columns} + \text{Rows})/2$:

- **Base stations:** Defaults to $\text{round}(0.10 \times M)$ with an upper limit of $\text{round}(0.50 \times M)$.
- **Obstacles:** Defaults to $\text{round}(0.25 \times M)$ with an upper limit of $\text{round}(1.00 \times M)$.
- **Rescuers:** At least one rescuer per station, up to a maximum of four per station ($\text{stations} \leq \text{rescuers} \leq 4 \times \text{stations}$). The default assigns one rescuer per station.
- **Pilots:** Bounded between $\max(\text{stations}, 9)$ and $4 \times \text{stations}$, ensuring that every station has at least one pilot and that each of the nine resource types has an assigned pilot.

6.3 Graphical User Interface

After configuration, the main application window is displayed (Figure 6). It is divided into two sections:

1. **Grid Canvas (Left):** A 2D grid rendering the environment, stationary structures, and mobile agents.

2. **Side Panel (Right):** Hosts runtime controls, the visual legend, and two real-time logging panes.

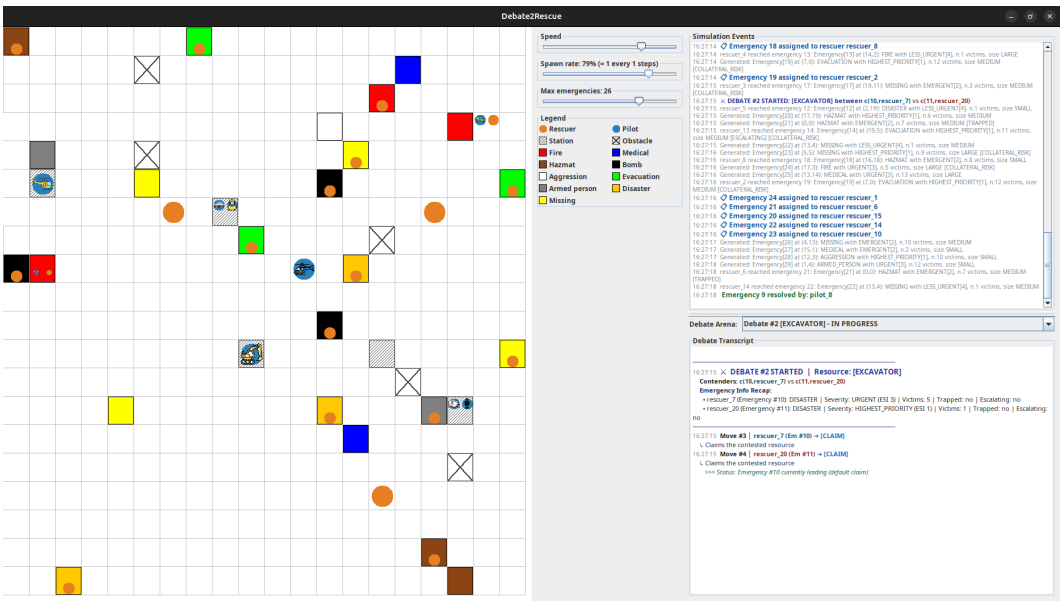


Figure 6: The main graphical user interface of Debate2Rescue.

6.3.1 Grid Elements

The canvas displays entities using distinct shapes and colors:

- **Base Stations:** Gray cells with diagonal stripes, representing bases where responders wait between assignments.
- **Obstacles:** White cells with a dark cross, representing impassable cells.
- **Emergencies:** Colored squares, color-coded by emergency category.
- **Responders:** Circles representing rescuer and pilot agents, color-coded by role.

A visual legend at the bottom-left of the side panel maps each icon and color to its corresponding entity or emergency type.

6.3.2 Cell Tooltips

Hovering the cursor over any station or emergency cell displays a tooltip with details such as emergency status, severity level (and ESI score), victim count, station presence, and any responders currently on the cell.

6.4 Runtime Simulation Controls

Three sliders at the top of the side panel allow adjusting simulation parameters dynamically during execution (Figure 7):

- **Speed:** Controls step execution delay; moving the slider to the right increases simulation speed.
- **Spawn rate:** Controls the per-step probability of spawning a new emergency (0% to 100%). Setting it to 0% disables new emergency generation.
- **Max emergencies:** Sets the upper limit on concurrent active emergencies in the grid.



Figure 7: Runtime adjustment sliders in the side panel.

6.5 Real-Time Logging

The side panel features two vertically split logging panes: the upper pane logs general simulation events, while the lower pane streams debate transcripts.

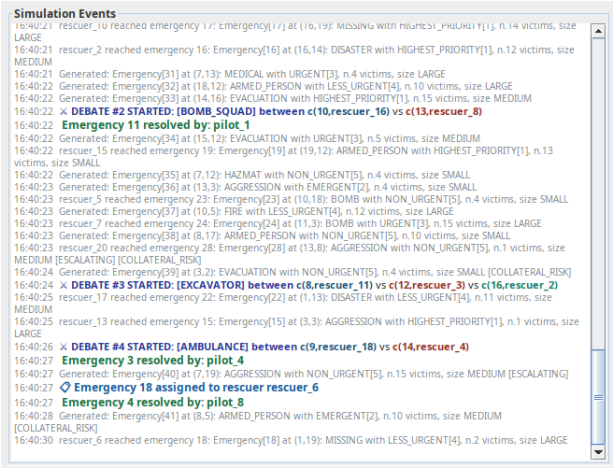


Figure 8: Simulation events log (top pane).

6.5.1 Simulation Events Pane

The upper pane logs environment and multi-agent lifecycle events in real time (Figure 8):

- **Routine grid events:** Actions already visible on the canvas (agent movement, emergency spawn, agent registration) are shown in muted gray to reduce clutter.
- **Task coordination events:** Highlighted with icons and colors:
  - `[Assign]` (blue): Emergency assignment sent to a rescuer.
  - `[Resolve]` (green): Emergency successfully resolved.
  - `[Message]` (dark text): Direct inter-agent communication.
- **Debate summaries:** One-line notices marking the start and conclusion of each resource debate, indicating the debate ID, contested resource, and winner.

6.5.2 Debate Arena Pane

The lower pane records the argumentation dialogue whenever two or more rescuers compete for the same specialized resource via Arg2P (Figure 9).

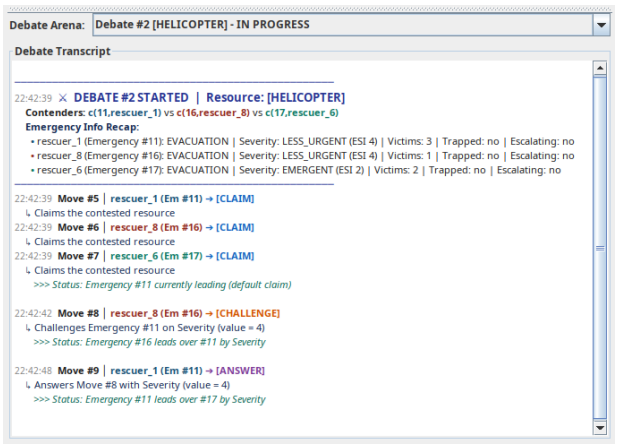


Figure 9: Debate arena transcript (bottom pane).

The Debate Arena contains:

- **Session selector:** A dropdown menu to switch between active and concluded debates (most recent first).
- **Header:** Summarizes the contested resource and contender emergency profiles (severity, victim count, trapped victims, escalation status).
- **Dialogue turns:** Color-coded by performative:
  - **CLAIM:** Resource request by an agent.

  - **CHALLENGE:** Attack on an opponent's priority based on criteria such as severity or victim count.
  - **ANSWER:** Response or counter-argument defending an emergency.
  - **CONCEDE:** Concession withdrawing a claim when an opponent's argument dominates.

- **Turn status:** Updates after each move to display the current leader and the applied ranking rule.
- **Debate summary:** Concluding banner showing the winning emergency, the assigned rescuer, the final justification, and any conceded contenders.

6.6 Persistent File Logs

Each debate session is saved to disk for inspection and offline analysis.

6.6.1 Directory Structure and File Naming

On each run, a timestamped directory is created under `logs/`:

`logs/yyyy-MM-dd_HH-mm-ss/`

Each debate produces an individual log file named:

`debate<ID>_<resource_type>.log`

For example, `debate1_fire_truck.log`.

6.6.2 Log File Structure

Each log file contains three parts:

1. **Header:** Start timestamp, contested resource, and contender profiles (emergency ID, severity, victim count, trapped victims, and escalation status).
2. **Moves log:** Timestamped record of each argumentative move and the intermediate leading state with its ranking rule.
3. **Footer:** Final decision, winner, conceded contenders, deciding justification, and total move count.

A sample debate log is shown in Figure 10.

```
================================================================
DEBATE #1 STARTED: Resource Type [fire_truck]
Timestamp: 2026-10-28 14:35:45
Contenders: c(7,rescuer_15), c(10,rescuer_13)

Contender Emergency Infos:
  * rescuer_15 (Emergency #7): FIRE | Severity: LESS_URGENT (ESI 4)
    | Victims: 12 | Trapped: no | Escalating: no
  * rescuer_13 (Emergency #10): FIRE | Severity: EMERGENT (ESI 2)
    | Victims: 2  | Trapped: no | Escalating: no
================================================================

--- DEBATE MOVES & TURNS ---
[2026-10-28 14:35:45] Move #1: rescuer_15 (Emergency 7)  -> CLAIM: claim
[2026-10-28 14:35:45] Move #2: rescuer_13 (Emergency 10) -> CLAIM: claim
       >>> Status: Leading Emergency = 7 | Reason: []

[2026-10-28 14:35:54] Move #5: rescuer_13 (Emergency 10) -> CHALLENGE:
       challenge(7,severity,2)
       >>> Status: Leading Emergency = 10 | Reason:
       [rank(severity,10,7),rank(critical,10,7)]

[2026-10-28 14:36:07] Move #9: rescuer_15 (Emergency 7) -> CONCEDE:
       "withdrew claim"
       >>> Status: Leading Emergency = 10 | Reason: []

================================================================
DEBATE #1 CONCLUDED: Resource Type [fire_truck]
Timestamp: 2026-10-28 14:36:07
FINAL DECISION: Winner is Emergency 10
DENIED CONTENDERS: c(7,rescuer_15) [conceded]
JUSTIFICATION: All opponent(s) conceded (deciding argument:
       [rank(severity,10,7),rank(critical,10,7)])
TOTAL MOVES: 4
================================================================
```

Figure 10: Excerpt of a persistent debate log file (`debate1_fire_truck.log`).

7 Conclusion

This project has been challenging because it made us think of the system as a group of autonomous agents that negotiate, instead of a program that computes an allocation. A large part of the effort went into Arg2P, a tool none of us knew, since the course did not cover it. Its syntax is compact, but it took us time to understand how rules and preferences interact under the grounded labelling, and to write a theory whose labels say something about the domain.

The theory in section 5.2 went through several versions, becoming more and more Arg2P-centered. The first one had only claims, so the oldest emergency always won. In the second, a rescuer could object to another claim by naming a reason, and a single Prolog predicate decided whether the objection counted, checking that the reason put the rescuer ahead and that it was stronger than the reason it answered. Arg2P only propagated the outcome, and the rescuers kept their own copy of the order of the reasons to pick their moves. Now Prolog only tells which of two values is better for the same reason. The strength of the reasons became a preference between rankings, so Arg2P is the one that weighs a reason against another, and the rescuers ask it which move would help them through `arg.choose`. Then, we enriched the debates with exceptions.

Overall, we are satisfied with our work and how debates are carried on. The winner comes out of the labelling computed by Arg2P, and the orchestrator falls back to the oldest emergency only when no claim is accepted. Responsibilities are divided between the orchestrator, which coordinates the turns and assigns the pilot, and each rescuer, which decides on its own which value to reveal. The orchestrator only sees where the emergencies are and each rescuer only sees its own, so both learn the values of the others from the moves.

7.1 Future Work

- **Faster debates.** Every move rebuilds the whole theory and runs Arg2P from scratch, in the orchestrator and in each hypothetical debate tried by `arg.choose`, and our cache only helps when the same theory comes back. An incremental evaluation would avoid most of this work.

- **Richer debates.** Rescuers can only reveal values or concede, and nobody can question a revealed value. Letting a rescuer doubt the severity declared by another one would need premises that can be attacked.

- **Fewer debates.** With N free pilots and more requests, the orchestrator runs N debates in a row with one winner each. A theory that grants more than one claim could serve N emergencies in a single debate.

- **A single copy of the rules.** `rescuer.asl` repeats which values count as a reason, and `arg.choose` hard-codes the pairs of values that complete a situation, while both could be read from `debate.pl`.

References

Rafael H. Bordini, Jomi Fred Hübner, and Michael Wooldridge. *Programming Multi-Agent Systems in AgentSpeak using Jason*. John Wiley & Sons, 2007.

Roberta Calegari, Andrea Omicini, Giuseppe Pisano, and Giovanni Sartor. Arg2P: an argumentation framework for explainable intelligent systems. *Journal of Logic and Computation*, 32(2):369–401, 2022. doi: 10.1093/logcom/exab089.

Phan Minh Dung. On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2):321–357, 1995. ISSN 0004-3702. doi: 10.1016/0004-3702(94)00041-X.

Sanjay Modgil and Henry Prakken. The ASPIC+ framework for structured argumentation: a tutorial. *Argument & Computation*, 5(1):31–62, 2014. doi: 10.1080/19462166.2013.869766.

Giuseppe Pisano, Roberta Calegari, Andrea Omicini, and Giovanni Sartor. A mechanism for reasoning over defeasible preferences in Arg2P. In Stefania Monica and Federico Bergenti, editors, *Proceedings of the 36th Italian Conference on Computational Logic (CILC 2021)*, volume 3002 of *CEUR Workshop Proceedings*, pages 16–30, 2021. URL `https://ceur-ws.org/Vol-3002/paper10.pdf`.

Iyad Rahwan, Sarvapali D. Ramchurn, Nicholas R. Jennings, Peter McBurney, Simon Parsons, and Liz Sonenberg. Argumentation-based negotiation. *The Knowledge Engineering Review*, 18(4):343–375, 2003. doi: 10.1017/S0269888904000098.